

Progetto Finale: Pipeline di Parsing Web e Valutazione con LLM-as-Judge

Denis Fava
Matricola 2018687

Riccardo Pucci
Matricola 1994172

Alessandro Cantaressi
Matricola 2129091

1 Obiettivo del progetto

Il sistema realizza una pipeline completa per l'acquisizione, l'estrazione e la valutazione di contenuto informativo da pagine web appartenenti a quattro domini eterogenei: `en.wikipedia.org`, `www.basketball-reference.com`, `global.morningstar.com` e `it.tradingview.com`.

Data una URL, il sistema scarica la pagina, ne estrae il testo informativo producendo una versione pulita in Markdown, e ne misura la qualità in due modi indipendenti: con metriche automatiche calcolate rispetto a un Gold Standard costruito a mano, e con un giudizio qualitativo prodotto da un LLM locale. Tutto è esposto tramite API REST e tramite una Web UI, ed è orchestrato in quattro container avviabili con un solo comando.

La scelta dei quattro domini non è casuale ed è la ragione per cui il progetto è interessante: un'enciclopedia fatta di prosa, un archivio statistico fatto di tabelle, un sito editoriale finanziario e una single page application che costruisce metà pagina in JavaScript pongono problemi di parsing strutturalmente diversi. Come si vedrà nella sezione 4, mettono in crisi anche le metriche di valutazione, ed è lì che il confronto fra metrica automatica e giudizio del modello diventa istruttivo.

2 Organizzazione del codice

Il progetto è orchestrato da `docker-compose.yaml` e si compone di quattro servizi: **backend** (porta 8003), **frontend** (8004), **mariadb** (3306) e **ollama** (11434). Nessun servizio richiede passaggi manuali dopo `docker compose up --build`: lo schema del database viene creato all'inizializzazione del container MariaDB, il modello LLM viene scaricato dall'endpoint del container Ollama, e il backend popola il database leggendo i file JSON di `gs_data/`.

I parser e la programmazione a oggetti

I quattro parser condividono lo stesso ciclo di vita e sono organizzati come gerarchia di classi. **BaseParser** è una classe astratta che fissa la sequenza una volta sola, secondo il pattern *template method*:

1. **fetch** — ottiene l'HTML, dal database in fase di valutazione o dalla rete altrimenti;
2. **make_soup** — costruisce l'albero DOM;
3. **prepare** — rimuove il rumore specifico del sito;
4. **extract_title** — estrae il titolo;
5. **extract_blocks** — estrae i blocchi informativi;
6. **finalize** — compone il Markdown finale.

Solo `extract_blocks` è astratto: è l'unico passo davvero specifico di ciascun sito. Le quattro sotto-classi ridefiniscono i restanti metodi soltanto quando serve, e il vantaggio si misura in concreto: aggiungere un dominio significa scrivere una sottoclasse e una riga nel registro, mentre la gestione dell'HTML vuoto, la configurazione di `Crawl4AI` e il formato del risultato arrivano dalla classe base, identici per tutti.

Tre ridefinizioni meritano una nota, perché documentano altrettante scoperte fatte sul campo:

- **WikipediaParser.fetch** usa una richiesta HTTP semplice invece di `Crawl4AI`. Le pagine di Wikipedia sono renderizzate lato server: aprire un browser headless produrrebbe lo stesso HTML impiegando secondi in più.
- **BasketballReferenceParser.prepare** riporta nel DOM le tabelle che il sito nasconde dentro commenti HTML e ricompone in JavaScript. È anche il motivo per cui il fetch restituisce `result.html` e non `cleaned_html`: quest'ultimo elimina i commenti, e con essi metà del contenuto.
- **BasketballReferenceParser.parse** è l'unica sottoclasse che ridefinisce il `template method`, perché la bacheca dei premi (`<ul id="bling">`) va letta fra la preparazione e la pulizia, in un punto che la sequenza standard non prevede.

Il dispatch avviene in `parsers/registry.py`, unico punto in cui un dominio viene associato al proprio parser: `server.py` non contiene alcun ramo condizionale sul dominio.

I servizi e il livello REST

La logica applicativa sta in `backend/src/services/`: `database.py` (connessione e pool), `repository.py` (tutte le query, e solo lì), `evaluator.py` (metriche), `judge.py` (dialogo con Ollama), `bootstrap.py` (inizializzazione), `text_utils.py` e `markdown_utils.py` (normalizzazione).

`server.py` resta un livello sottile di orchestrazione e validazione: espone i quattordici endpoint richiesti, ciascuno con il proprio `response_model` Pydantic, e traduce gli errori dei livelli sottostanti in codici HTTP. Il frontend, in Jinja2, dialoga esclusivamente con le API REST e non conosce il database.

3 Valutazione dei parser

Il Gold Standard contiene **40 entry**, dieci per dominio, ciascuna con URL, dominio, titolo, HTML grezzo e testo di riferimento copiato a mano dal browser. La valutazione lavora sempre sull'HTML statico salvato

nel database: è ciò che la consegna richiede, ed è anche l'unico modo di ottenere numeri confrontabili nel tempo su siti che cambiano ogni giorno.

Sono implementate due metriche. `token_level_eval` è quella obbligatoria e confronta gli *insiemi* di token. `sequence_eval` è stata aggiunta da noi e lavora sulle *sequenze*: calcola precisione, recall e F1 sulla più lunga sottosequenza comune, con l'algoritmo di `diff`lib.`SequenceMatcher`.

La seconda esiste perché la prima ha un punto cieco preciso. Lavorando su *insiemi*, ordine e ripetizioni non contano: un parser che restituisse le stesse parole in ordine casuale, o che ripetesse tre volte lo stesso paragrafo, otterrebbe un F1 identico a quello di un'estrazione corretta. Abbiamo verificato il caso limite: un testo con un paragrafo ripetuto tre volte ottiene token F1 pari a 1.000 e sequence F1 pari a 0.737.

Dominio	token F1	seq. F1	judge
morningstar.com	0.965	0.951	5.00
basketball-reference.com	0.949	0.922	3.20
wikipedia.org	0.930	0.951	4.80
tradingview.com	0.889	0.890	2.80

Tabella 1: Metriche medie per dominio, su dieci pagine ciascuno.

Tutti e quattro i domini superano la soglia di 0.80 che la consegna classifica come “Buono”. Il dominio più difficile è TradingView, e la ragione è strutturale: le sue schede sono griglie di dati generate da React, con nomi di classe CSS che contengono un hash di build e cambiano a ogni rilascio. Il parser si aggancia perciò agli attributi `data-qa-id`, che il sito usa per i propri test automatici e che restano stabili, e adotta una whitelist esplicita dei widget informativi anziché una blacklist del rumore.

Due osservazioni emerse durante la costruzione del Gold Standard, che riteniamo la parte più istruttiva del lavoro.

La prima riguarda la **coerenza del criterio di copiatura**. Su Basketball-Reference, quattro pagine su dieci erano state trascritte includendo le tabelle statistiche stagionali, che il parser esclude di proposito, e omettendo la coda della pagina, che il parser include. Il risultato era un F1 medio di 0.678 con singoli valori fino a 0.096: non un difetto del parser, ma una divergenza di criterio fra chi aveva scritto il testo di riferimento e chi aveva scritto il codice. Uniformato il criterio, il dominio è passato a 0.949.

La seconda riguarda i **caratteri invisibili**. Il testo copiato da un browser trascina con sé marcatori Unicode di direzionalità e byte order mark, che TradingView inserisce attorno a ogni valore numerico. Il parser li rimuove, il testo incollato no, e poiché la metrica confronta token separati da spazio, lo stesso numero risultava diverso da sé stesso. La normalizzazione è stata collocata in `text_utils.normalize_gold_text`, invocata nell'unico punto in cui un `gold_text` entra nel database — e non dentro la metrica: così nel database esiste una sola versione del testo, ed è quella pulita.

4 Valutazione dei judge

Il judge dialoga con Ollama via REST su `/api/chat`, con `"stream": false`. Le responsabilità sono separate fra i due messaggi: il messaggio `system` contiene ruolo, criteri di penalizzazione e scala di punteggio; il messaggio `user` contiene solo i due testi da confrontare. Il formato è imposto a monte con `"format": "json"`, la temperatura è bassa perché serve un giudizio ripetibile, e i testi vengono troncati a 2000 caratteri — consentito dalla consegna, e su CPU la differenza fra una valutazione che termina e una che va in timeout.

Il fallback è comunque implementato, come la consegna richiede in modo esplicito: se il modello non produce JSON leggibile o i campi attesi mancano, il sistema restituisce punteggio minimo con `parse_ok` a falso. La distinzione conta, perché la percentuale di risposte valide è uno dei criteri con cui abbiamo scelto il modello.

Modello	media	JSON	s	corr.
ministral-3:3b	4.17	100%	13.1	−0.926
llama3.2:3b	4.00	100%	7.6	−0.676
phi4-mini	3.33	100%	18.6	−0.621
qwen3:4b	1.00	0%	27.8	—
gemma4:e2b	1.00	0%	30.7	—

Tabella 2: Confronto fra i cinque modelli ammessi. “corr.” è la correlazione di rango fra il punteggio del judge e l’F1; il punteggio 1.00 dei due modelli senza JSON valido è il valore di fallback, non un giudizio.

Due modelli su cinque non hanno prodotto una singola risposta valida, ed erano anche i due più lenti. Sono entrambi modelli che generano token di ragionamento prima della risposta, e il limite di generazione impostato li troncava prima che arrivassero al JSON: un esempio concreto del fatto che “il modello non funziona” e “il modello è configurato male” si assomigliano molto dall'esterno. Il punteggio 1.00 che compare per entrambi non è un giudizio, è il valore di fallback contato cinque volte su cinque.

Abbiamo scelto **ministral-3:3b**: punteggio medio più alto, piena affidabilità sul formato e la correlazione più netta con la metrica automatica, accettando in cambio di essere più lento di **llama3.2:3b**.

La colonna della correlazione merita una lettura attenta, perché il segno è *negativo* per tutti e tre i modelli affidabili, e per il migliore vale −0.926. Non significa che il judge sbaglia: significa che ordina le pagine quasi esattamente al contrario di come le ordina l’F1. È la stessa contraddizione visibile nella tabella 1, qui misurata invece che osservata.

Lo stesso disaccordo si legge nella tabella 1, in forma più sfumata perché lì i valori sono medie su dieci pagine. Basketball-Reference ha il secondo miglior F1 (0.949) ma il secondo peggior judge (3.20), mentre Wikipedia, con un F1 più basso (0.930), ottiene 4.80.

La spiegazione è che le due metriche rispondono a domande diverse. La metrica a token misura *quanto* il parser ha estratto rispetto al riferimento; il judge valuta *se il risultato somiglia a un testo informativo*. Su

pagine che sono griglie di dati — una scheda titolo, un roster di squadra — la seconda domanda ha una risposta strutturalmente bassa anche quando l'estrazione è corretta, perché il modello legge un elenco di numeri e non riconosce prosa. Non a caso i due domini con il judge più basso — TradingView e Basketball-Reference — sono i due fatti di tabelle, e i due con il judge più alto sono quelli fatti di testo scritto.

È esattamente il motivo per cui la consegna richiede entrambe le valutazioni: nessuna delle due, da sola, descriverebbe il sistema. E la conseguenza pratica è che un judge non va usato come sostituto della metrica, ma come secondo parere su una domanda che la metrica non pone.

5 Organizzazione del database

Il database è MariaDB, interrogato con MariaDB Connector/Python. Tutte le query stanno in `repository.py` e usano esclusivamente segnaposto `?` con tupla di valori: nessuna query è costruita per interpolazione di stringa, che è la porta d'ingresso classica della SQL injection.

Lo schema, definito in `mariadb_data/init.sql` ed eseguito automaticamente alla creazione del container, comprende cinque tabelle. `web_resources` e `gold_standard` hanno nome e colonne fissati dalla consegna e sono legate da una relazione uno-a-uno: la stessa colonna `url` è insieme chiave primaria e chiave esterna, con `ON DELETE CASCADE` a implementare il requisito per cui `DELETE /web_resource` deve rimuovere anche il gold standard associato. Le altre tre — `parsed_documents`, `evaluations`, `judgements` — sono di nostra progettazione e servono a `/db_stats`, che per specifica deve leggere dati già calcolati invece di rifare il lavoro a ogni chiamata.

Un dettaglio implementativo merita di essere riportato perché non è ovvio. InnoDB limita una chiave a 3072 byte; in `utf8mb4` un carattere occupa fino a quattro byte, quindi una `VARCHAR(2048)` come chiave primaria supererebbe il limite e la `CREATE TABLE` fallirebbe. Poiché gli URL sono ASCII per definizione (RFC 3986 impone il percent-encoding per tutto il resto), la colonna è dichiarata `CHARACTER SET ascii COLLATE ascii_bin`: 2048 byte, sotto il limite, con la lunghezza richiesta dalla specifica invariata e in più un confronto case-sensitive, che per un URL è il comportamento corretto.

Infine, una scelta di architettura dei dati che l'esperienza ci ha imposto. I file JSON di `gs_data/` sono la fonte di verità e il database è una copia di lavoro, ripopolabile da zero a ogni avvio. La ragione è che i dati del sistema non sono tutti uguali: il parsing e le metriche si possono ricalcolare in qualunque momento, un testo di riferimento scritto a mano no. Trattare diversamente ciò che è riproducibile da ciò che non lo è non è un dettaglio organizzativo, ed è anche il motivo per cui la consegna chiede che i file JSON facciano parte del progetto consegnato.